

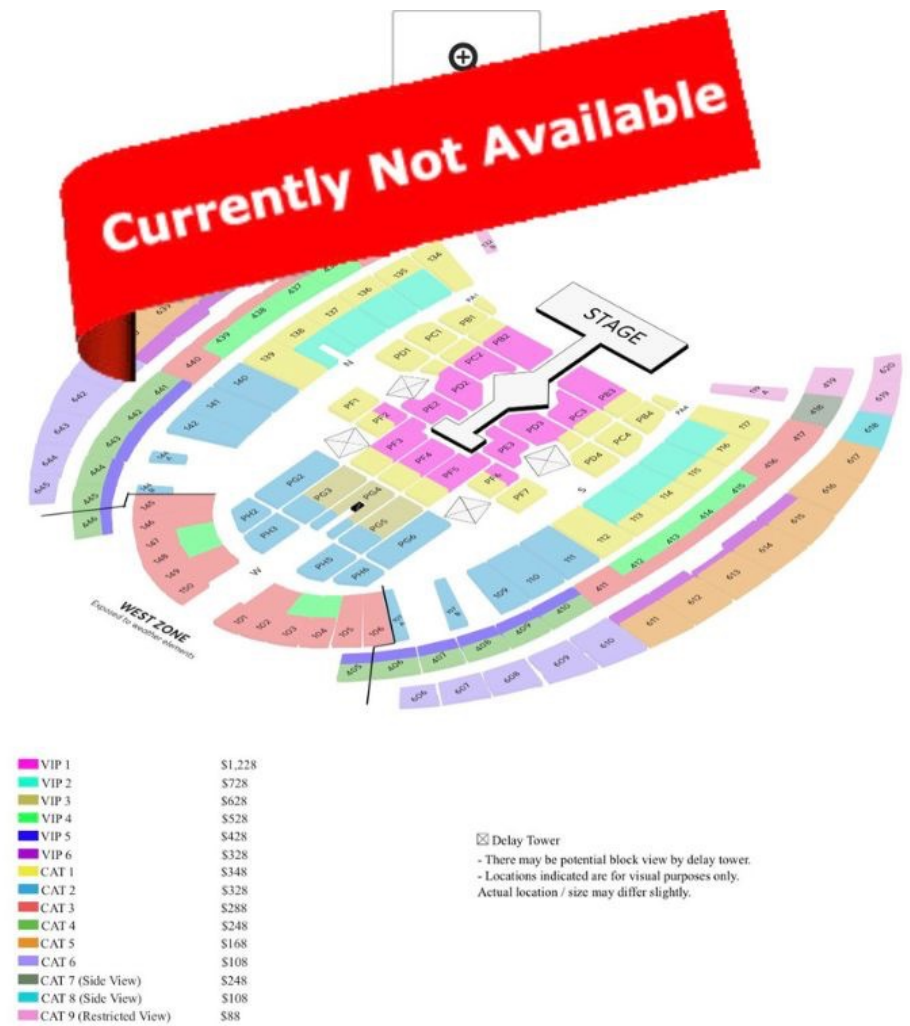
Cow book (3rd edition): Chapters 16,17

TRANSACTIONS AND CONCURRENCY CONTROL

Buying concert tickets

Buying tickets to the next Taylor Swift concert:

- Fill up form with your email address
 - Put in credit card number
 - Click submit
 - Cell phone message shows money deducted from your account...
- ... and then this happens



Changes to the database should be ALL or NOTHING !

Windows

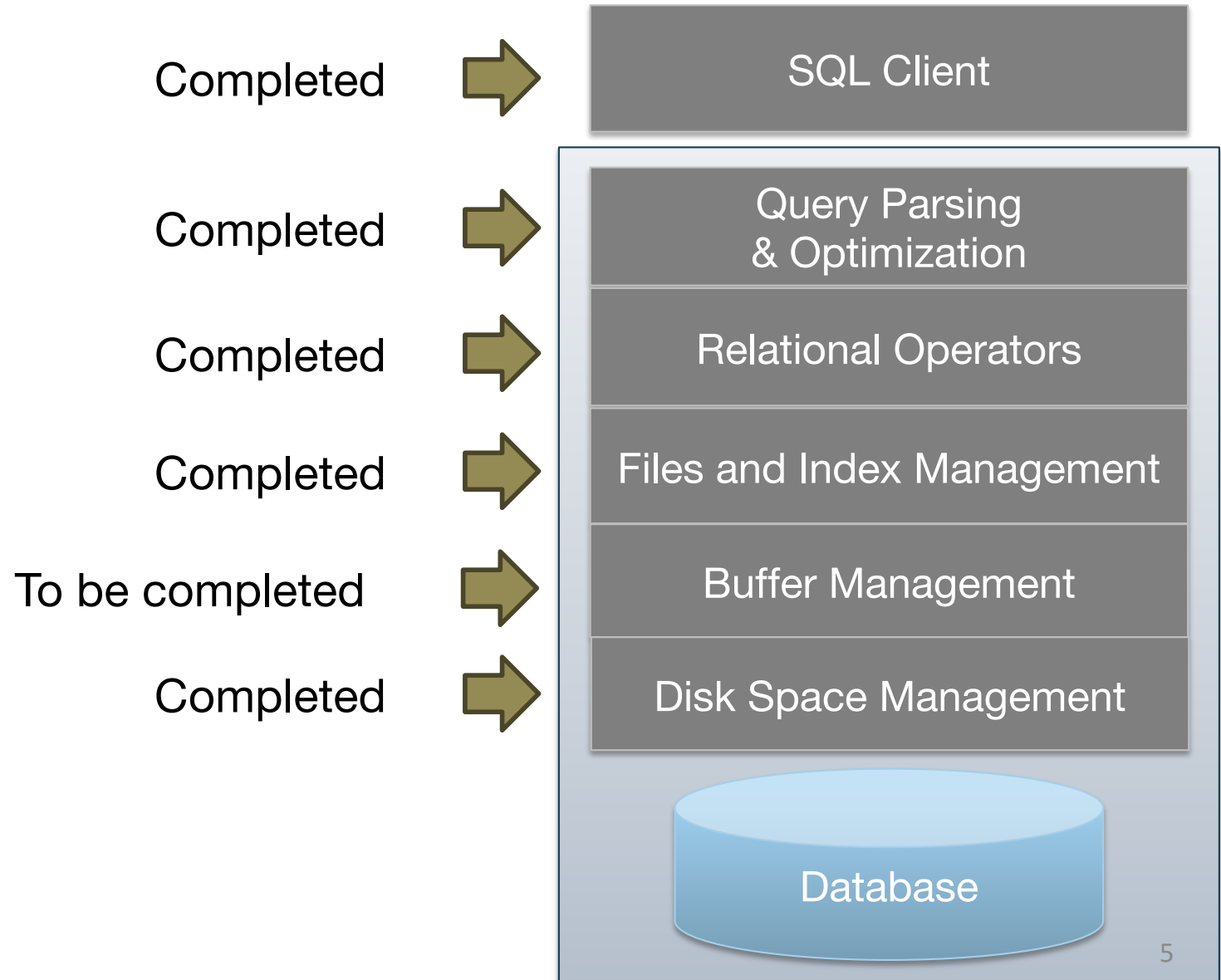
A fatal exception 0E has occurred at 0137:BFFA21C9. The current application will be terminated.

- * Press any key to terminate the current application.
- * Press CTRL+ALT+DEL again to restart your computer. You will lose any unsaved information in all applications.

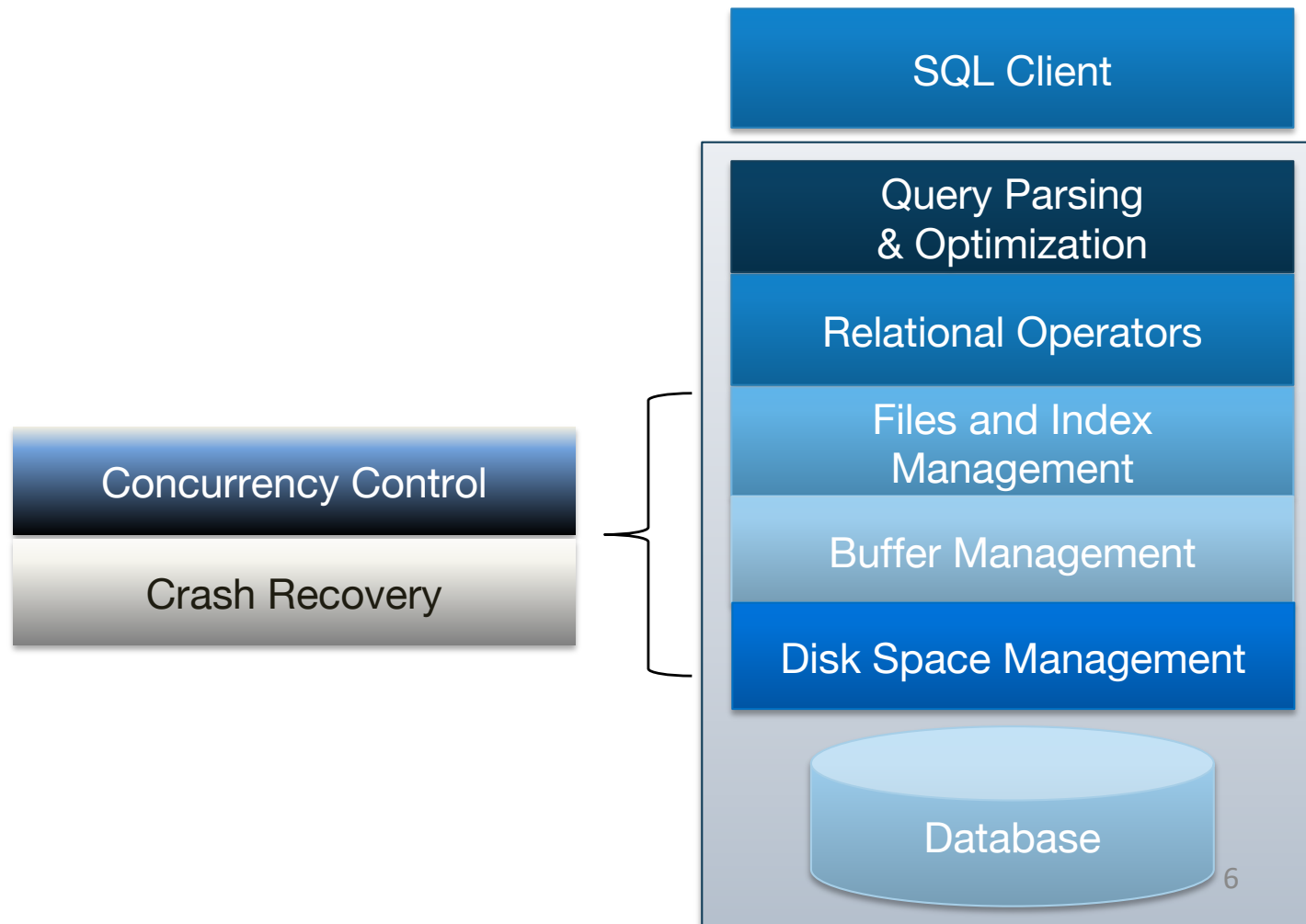
Press any key to continue _

Changes to the database should be ALL or NOTHING !

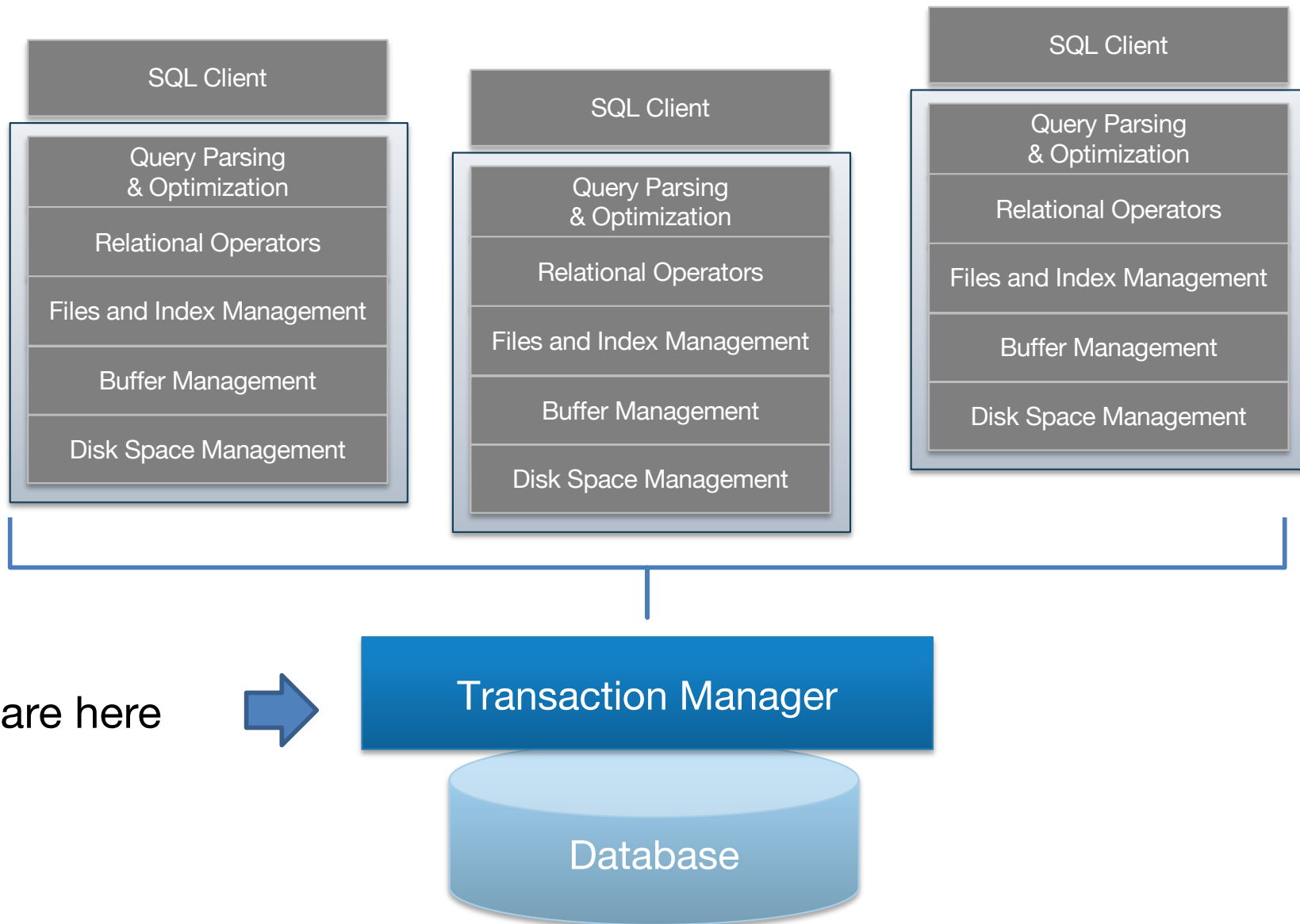
Architecture of an RDBMS



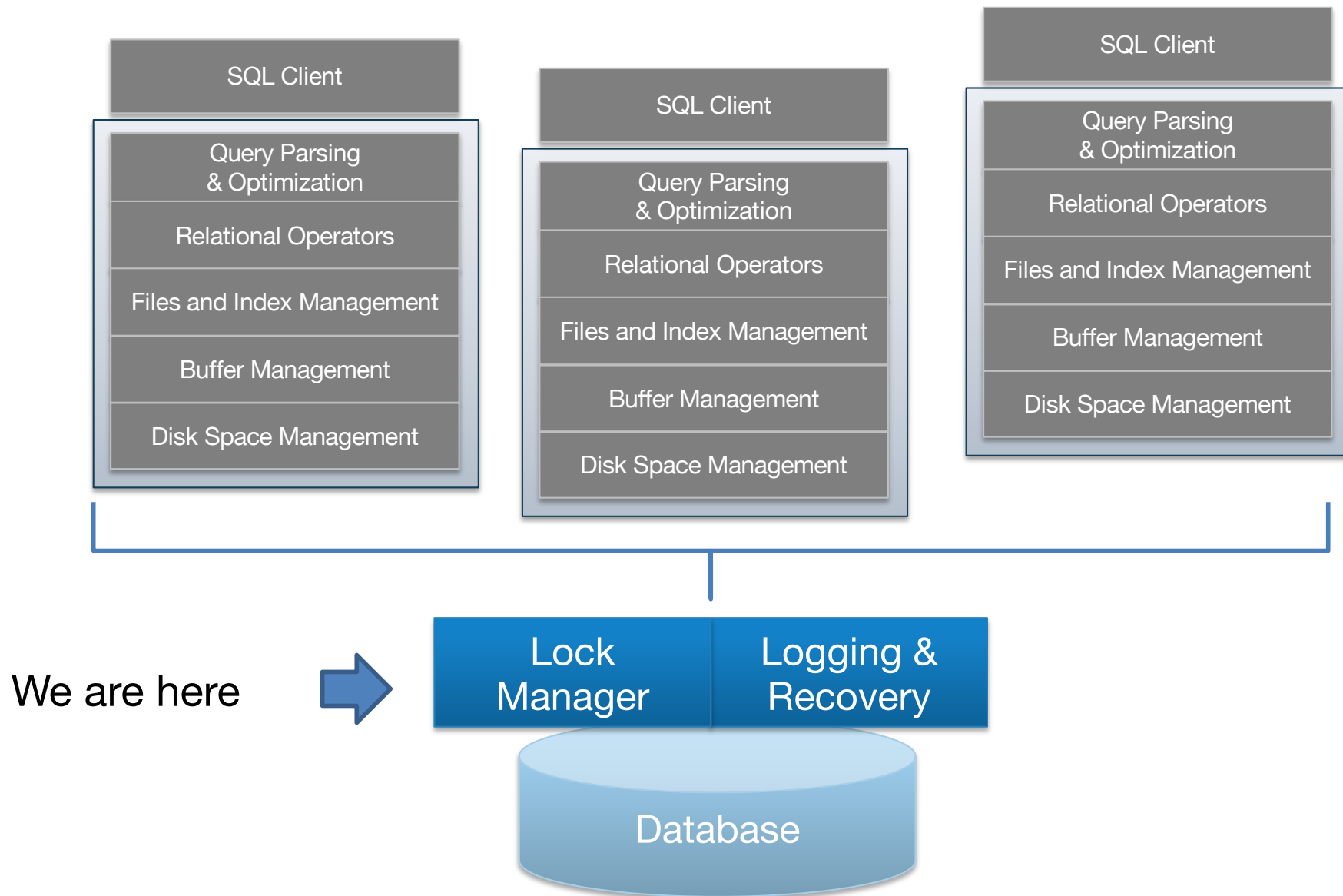
Remember this slide...



Architecture of an RDBMS



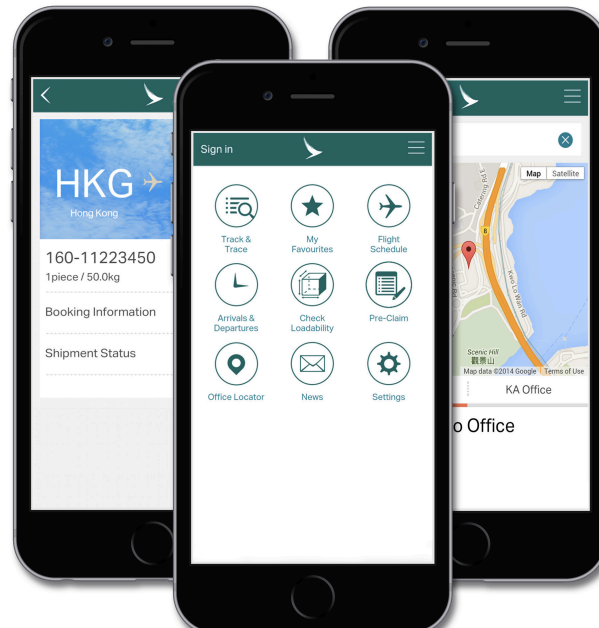
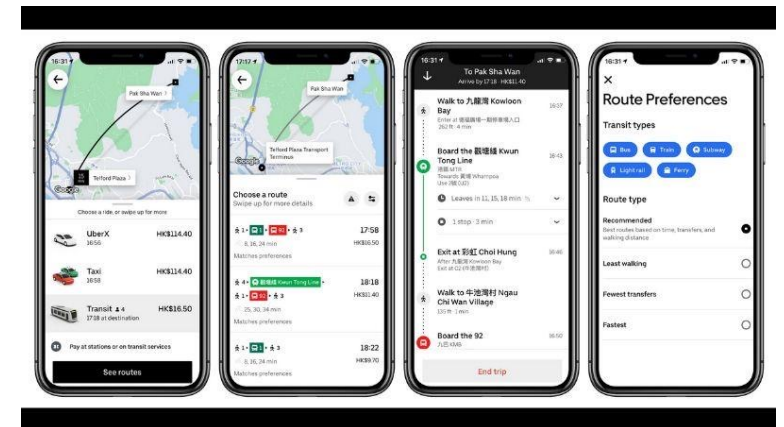
Architecture of an RDBMS



Applications on an RDBMS

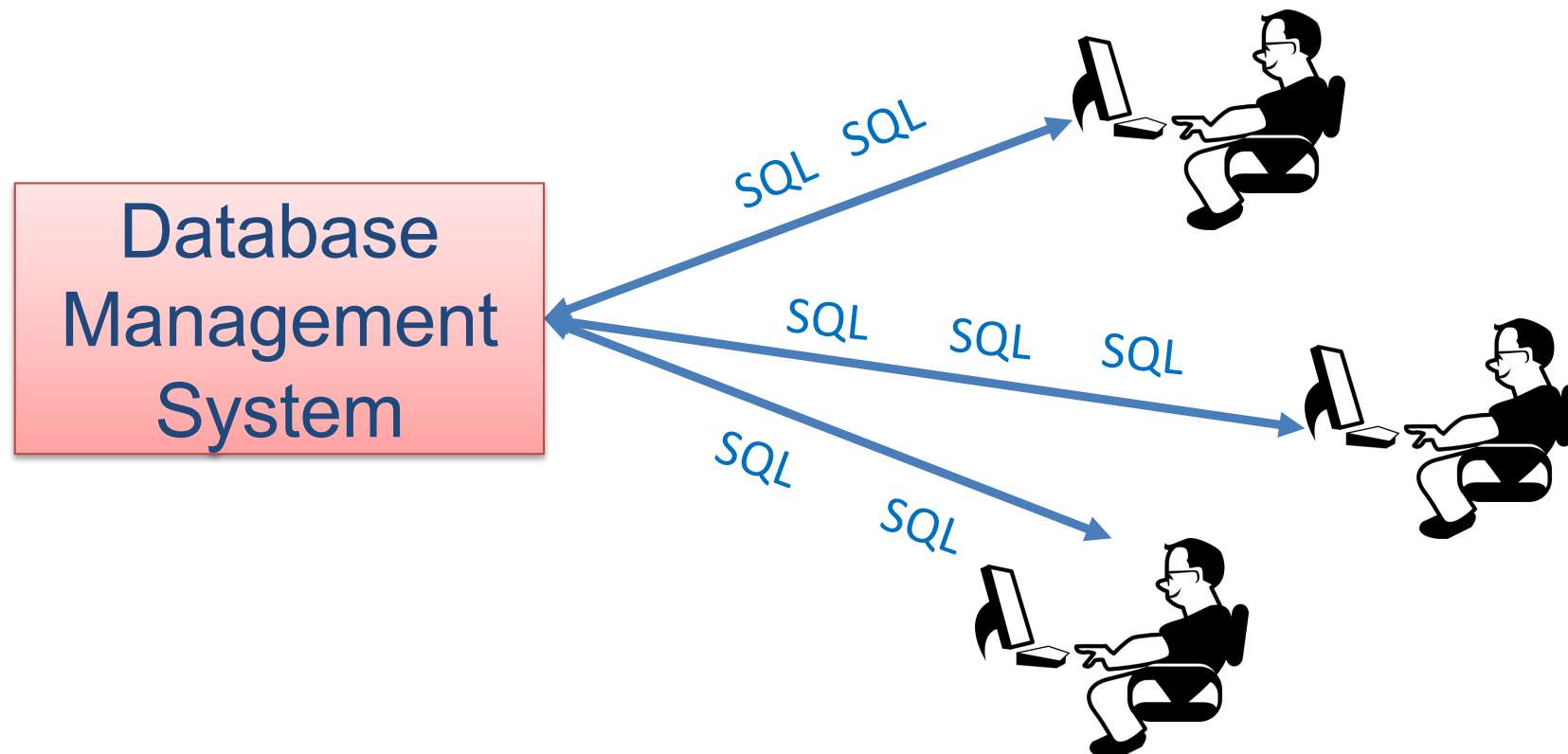
Any compute service that maintains state today is an application on top of some kind of DBMS.

- Uber
- Cathay Pacific
- Amazon.com
- HSBC Bank
- LeaveHome Safe



Applications want something from the RDBMS

- Queries and updates of course: what you learned so far!
- Real applications are composed of many statements being generated by user behaviors
- Many users work with the application at the same time



Concurrency control & recovery

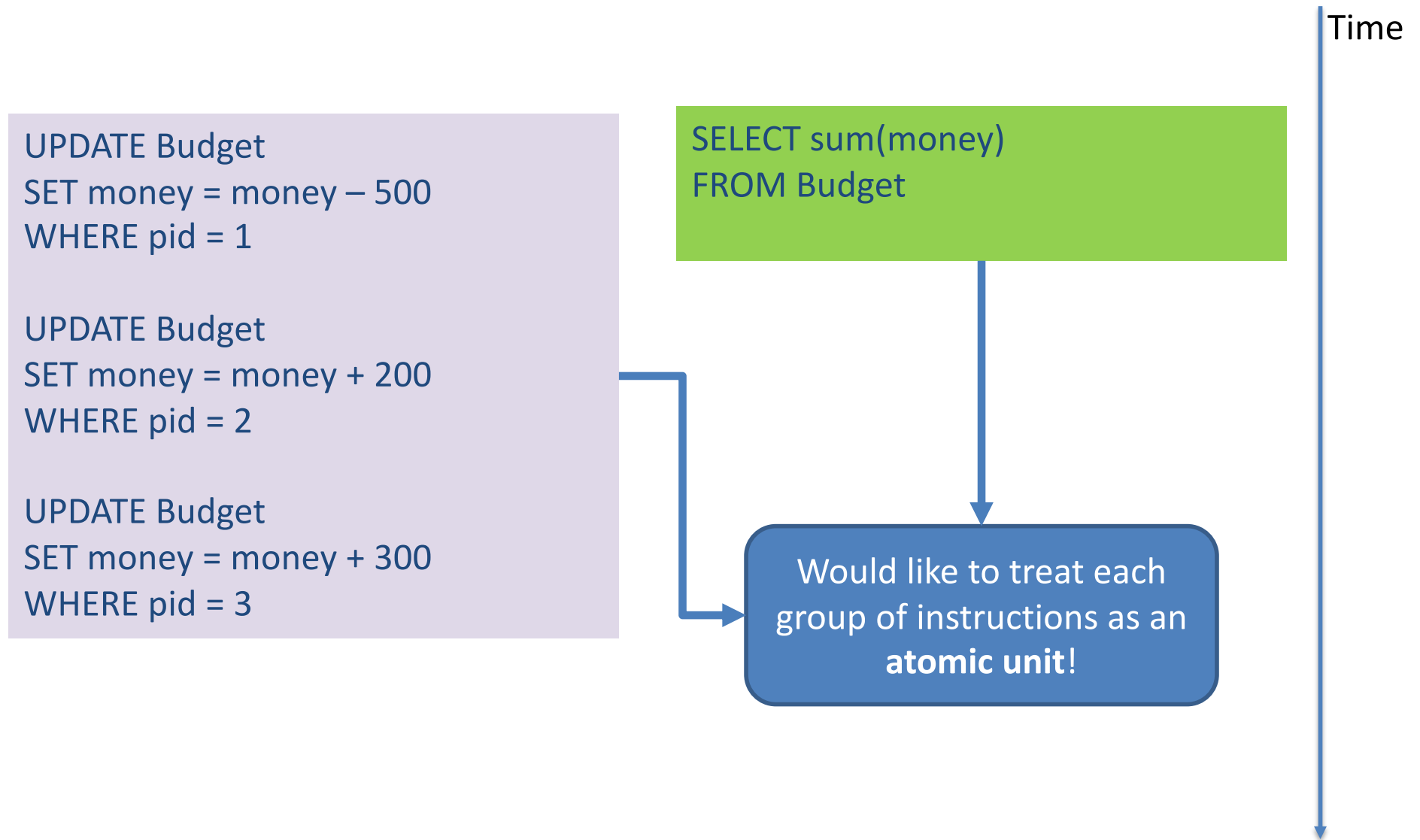
- **Concurrency control**
 - Provide correct and fast data access in the presence of concurrent work by many users
 - Disorderly processing that provides the **illusion of order**
- **Recovery**
 - Ensures database is fault tolerant, and not corrupted by software, hardware or network failure
 - Storage guarantees for mission-critical data
- **It's all helping the programmer !**
 - Providing guarantees that can lighten the load of app writers

Concurrent execution: why bother?

Multiple transactions are allowed to run concurrently in the system. Advantages are

- **Throughput:** Increased processor and disk *utilization* leads to more transactions per second (TPS) completed
 - Single core: e.g., one transaction can be using the CPU while another is reading from or writing to the disk
 - Multicore: ideally, scale throughput in the number of processors
- **Latency:** multiple transactions can run at the same time, so (with ample resources) one transaction's latency need not be dependent on another unrelated transaction
 - or that's the hope
- Both are important !

Example of concurrent execution



Different types of problems

User 1

```
INSERT INTO DollarProduct(name, price)
SELECT pname, price
FROM Product
WHERE price <= 0.99;
```

```
DELETE Product
WHERE price <= 0.99;
```

User 2

```
SELECT count(*)
FROM DollarProduct;
```

```
SELECT count(*)
FROM DollarProduct;
```

Time

What could go wrong?

Unrepeatable reads

Different types of problems



Different types of problems

User 1

User 2

```
INSERT INTO DollarProduct(name, price)
SELECT pname, price
FROM Product
WHERE price <= 0.99;
```

```
DELETE Product
WHERE price <= 0.99;
```

```
SELECT count(*)
FROM DollarProduct;
SELECT count(*)
FROM Product;
```

Time

What could go wrong?

Inconsistent reads

Different types of problems

User 1

```
UPDATE Product  
SET Price = Price - 10.99  
WHERE pname = "CoolToy" T1
```

User 2

```
UPDATE Product  
SET Price = Price * 0.6  
WHERE pname = "CoolToy" T2
```

Time

What could go wrong?

Race conditions and lost updates!

TRANSACTIONS

Transactions

- Major component of database systems
- Critical for most applications; arguably more so than SQL
- **Turing Awards to database researchers:**
 - Charles Bachman 1973 for pioneering early DBMS, including IDS
 - Edgar Codd 1981 for inventing relational DBs
 - Jim Gray 1998 for inventing transaction processing
 - Michael Stonebraker 2015 for pioneering relational DBMSs, including Ingres and Postgres

What is a transaction?

- Collection of operations that form a single logical unit
 - A sequence of actions considered to be one atomic unit of work
- Application view (SQL):
 1. Begin transaction
 2. Set of SQL statements
 3. End transaction
- **Examples**
 - Transfer money between accounts
 - Book a flight, a hotel, and a car together on Expedia

Our transaction model

- **Transaction:**
 - RDBMS's abstract view of an application program (or activity)
 - A sequence of **reads** and **writes** of database objects
 - Batch of work that must *commit* or *abort* as an atomic unit
- **Transaction manager controls execution of transactions**
- **Program logic is invisible to DBMS !**
 - Arbitrary computation possible on data fetched from the DB
 - The DBMS only see data read/written from/to the DB

(Note: some modern systems rethinking this assumption)

Transaction example

- **Transaction to transfer \$100 from account R to account S**

1. start transaction
2. read(R)
3. $R = R - 100$
4. write(R)
5. read(S)
6. $S = S + 100$
7. write(S)
8. end transaction

ACID guarantees

- **A tomicity:** **All** actions in the transaction happen, or **none** happen
- **C onsistency:** If the DB **starts** out **consistent**, it **ends** up **consistent** at the end of transaction
- **I solation:** Execution of **each transaction** is **isolated** from that of **other transactions**
- **D urability:** If a transaction **commits**, its effects **persist**

Atomicity and durability

- A transaction ends in one of two ways:
 - **Commit** after completing all its actions
 - “commit” is a contract with the caller of the DB
 - **Abort** (or be aborted by the DBMS) after executing some actions
 - Or **system crash** while transaction in progress; treat as abort
- Two key properties for committing / aborting transactions
 - **Atomicity**: Either execute all its actions, or none of them
 - **Durability**: Effects of a committed transaction must survive failures
- DBMS typically ensures the above by logging all actions:
 - **Undo** the actions of aborted / failed transactions.
 - **Redo** actions of committed transactions not yet propagated to disk when system crashes

Atomicity and durability (continued)

- **Atomicity**

- If the transaction fails after step 4 and before step 7, money will be “lost” → inconsistent database
- DBMS should ensure that updates of a partially executed transaction are cancelled

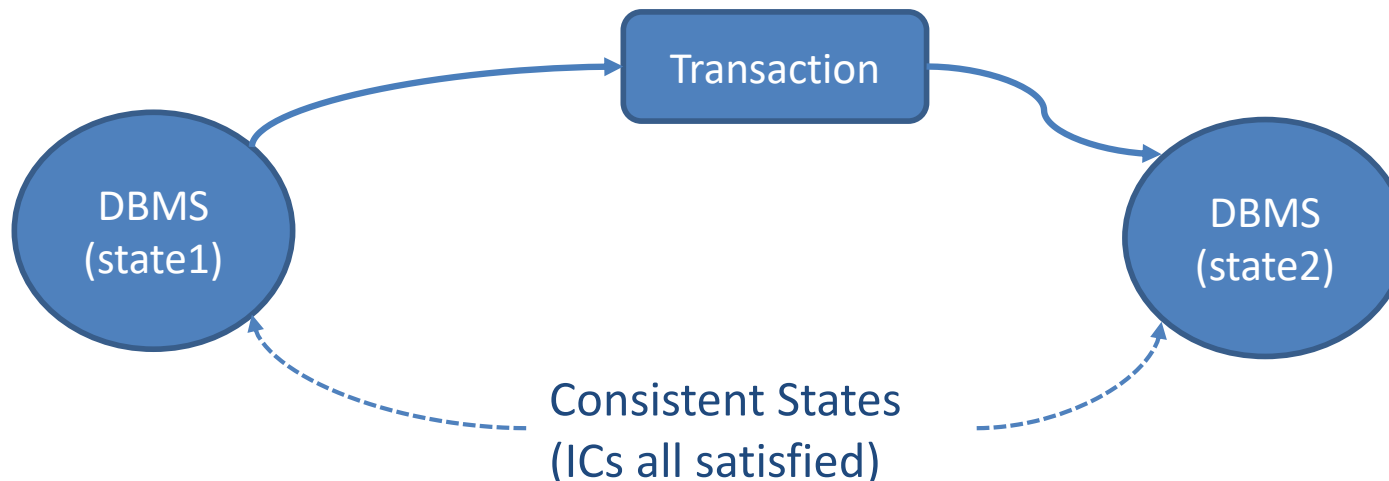
- **Durability**

- Once the user is notified that the transaction is complete, he/she should rest easy that \$100 were transferred from R to S.

1. start transaction
2. read(R)
3. $R = R - 100$
4. write(R)
5. read(S)
6. $S = S + 100$
7. write(S)
8. end transaction

Transaction consistency

- **Transactions preserve DB consistency**
 - Given a consistent DB state, produce another consistent DB state
- **DB consistency expressed as a set of declarative *integrity constraints***
 - CREATE TABLE / ASSERTION statements
- **Transactions that violate integrity are aborted**
 - That's all the DBMS can automatically check!



Isolation (concurrency)

- **DBMS interleaves actions of many transactions**
 - Actions = reads / writes of DB objects
- **DBMS ensures any 2 concurrent transactions do not “interfere”**
- **Each transactions executes as if ran by itself**
 - Concurrent accesses have no effect on transaction’s behavior (**no race conditions**)
- Isolation can be achieved easily by running one transaction at a time (**serially**)
 - However, serial execution it is not desirable (**why?**)
- **Net effect** must be identical to executing all transactions in **some (any) serial order**
- Users & programmers think about transactions in isolation
 - Without considering effects of other concurrent transactions !

Isolation (race condition example)

- **Isolation property (concurrency): no race conditions**
 - If another transaction T2 accesses R and S between steps 4 and 5 of T1, it will see a lower value for R+S. **Why?**

T1

1. start transaction
2. read(R)
3. $R = R - 100$
4. write(R)
5. read(S)
6. $S = S + 100$
7. write(S)
8. end transaction

T2

1. start transaction
2. read(R)
3. print(R+S)
4. end transaction

Quick quizz

Link the items in the two lists.

- | | |
|----------------|-------------------------------------|
| a) Atomicity | 1. Maintain integrity constraints |
| b) Consistency | 2. All or Nothing |
| c) Isolation | 3. Committed data survives failures |
| d) Durability | 4. No worry of race conditions |

Summary

- ACID transactions make guarantees that
 - Improve performance (via concurrency)
 - Relieve programmers of correctness concerns
 - Hide concurrency and failure handling!
- One key issue to consider, and mechanisms
 - Concurrency control (via two-phase locking / 2PL)
 - Recovery (via write-ahead logging / WAL)

CONCURRENCY CONTROL

Concurrency: providing isolation

- **Naïve approach - serial execution**
 - one transaction runs at a time
 - safe but slow
- **Execution of transactions must be interleaved for better performance**
- **With concurrent executions, how do we ensure correctness ?**

Scheduling transaction operations

Schedule: ordering of operations from various transactions that are to be executed concurrently.

Example: schedules for **T1** and **T2** (first two are **serial**)

1. start transaction
2. read(R)
3. $R = R - 100$
4. write(R)
5. read(S)
6. $S = S + 100$
7. write(S)
8. end transaction
9. start transaction
10. read(R)
11. print(R+S)
12. end transaction

1. start transaction
2. read(R)
3. print(R+S)
4. end transaction
5. start transaction
6. read(R)
7. $R = R - 100$
8. write(R)
9. read(S)
10. $S = S + 100$
11. write(S)
12. end transaction

1. start transaction
2. start transaction
3. read(R)
4. $R = R - 100$
5. write(R)
6. read(R)
7. read(S)
8. $S = S + 100$
9. print(R+S)
10. end transaction
11. write(S)
12. end transaction

Serializable schedules

We need a “touchstone” concept for correct behavior

- Definition: **Serial schedule (one transaction at a time)**
 - Each transaction runs from start to finish without any interfering actions from other transactions
- Definition: 2 schedules are **equivalent** if they:
 - involve the same transactions
 - each individual transaction's actions are ordered the same (right) way (**obvious**)
 - both schedules leave the DB in the same final state
- Definition: Schedule S is **serializable** if:
 - S is equivalent to **any** serial schedule

Schedule 1

- Say $A=B=200$ before T1 and T2
- Let T1 transfer \$100 from A to B and T2 add 10% interest to A & B
- Serial schedule in which T1 is followed by T2
- $A = ?$ $B = ?$

**T1: Transfer \$100
from A to B**

```
begin
read(A)
A = A - 100
write(A)
read(B)
B = B + 100
write(B)
commit
```

**T2: Add 10%
interest to A & B**

```
begin
read(A)
A = A * 1.1
write(A)
read(B)
B = B * 1.1
write(B)
commit
```

Schedule 1

- Say $A=B=200$ before T1 and T2
- Let T1 transfer \$100 from A to B and T2 add 10% interest to A & B
- Serial schedule in which T1 is followed by T2
- $A= 110$ $B= 330$

**T1: Transfer \$100
from A to B**

```
begin
read(A)
A = A - 100
write(A)
read(B)
B = B + 100
write(B)
commit
```

**T2: Add 10%
interest to A & B**

```
begin
read(A)
A = A * 1.1
write(A)
read(B)
B = B * 1.1
write(B)
commit
```

Schedule 2

- Say $A=B=200$ before T1 and T2
- Serial schedule in which T2 is followed by T1
- $A = ?$ $B = ?$

T1: Transfer \$100 from A to B	T2: Add 10% interest to A & B
-----------------------------------	----------------------------------

```
begin
read(A)
 $A = A * 1.1$ 
write(A)
read(B)
 $B = B * 1.1$ 
write(B)
commit
```

```
begin
read(A)
 $A = A - 100$ 
write(A)
read(B)
 $B = B + 100$ 
write(B)
commit
```

Schedule 2

- Say $A=B=200$ before T1 and T2
- Serial schedule in which T2 is followed by T1
- $A= 120$ $B= 320$

T1: Transfer \$100 from A to B	T2: Add 10% interest to A & B
-----------------------------------	----------------------------------

```
begin
read(A)
 $A = A * 1.1$ 
write(A)
read(B)
 $B = B * 1.1$ 
write(B)
commit
```

```
begin
read(A)
 $A = A - 100$ 
write(A)
read(B)
 $B = B + 100$ 
write(B)
commit
```

Schedule 3

- Say $A=B=200$ before T1 and T2
- Schedule in which actions of T1 and T2 are interleaved.
- $A = ?$ $B = ?$

**T1: Transfer
\$100 from A
to B**

**T2: Add 10%
interest to A
& B**

begin

read(A)

$A = A - 100$

write(A)

begin

read(A)

$A = A * 1.1$

write(A)

read(B)

$B = B + 100$

write(B)

commit

read(B)

$B = B * 1.1$

write(B)

commit

Schedule 3

- Say $A=B=200$ before T1 and T2
- Schedule in which actions of T1 and T2 are interleaved.
- $A = 110$ $B = 330$
- This is not a serial schedule but it is **equivalent** to schedule 1
 - Hence serializable!
- **Role of transaction manager (a.k.a scheduler): find such schedules !**

T1: Transfer
\$100 from A
to B

T2: Add 10%
interest to A
& B

begin

read(A)

$A = A - 100$

write(A)

begin

read(A)

$A = A * 1.1$

write(A)

read(B)

$B = B + 100$

write(B)

commit

read(B)

$B = B * 1.1$

write(B)

commit

Conflicting operations

- We need a more precise / easier check for equivalence than “leaves the DB in the same final state”
- Use notion of “conflicting” operations (read/write)
- **Definition: Two operations conflict if they**
 - are by different transactions
 - are on the same object (e.g, same record)
 - at least one of them is a write(Note: read-only transactions never conflict with one another)

Conflict serializable schedules

- **Definition: two schedules are conflict equivalent iff**
 - They involve the same actions of the same transactions, and
 - Every pair of conflicting actions is ordered the same way
- **Definition: schedule S is conflict serializable if**
 - S is conflict equivalent to some serial schedule
- **Note some serializable schedules are NOT conflict serializable**
 - A price we pay to achieve efficient enforcement

Conflict serializability - intuition

- **Alternative definition: a schedule S is conflict serializable if**
 - We are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A) W(A)

R(B) W(B)

R(A) W(A)

R(B) W(B)

Conflict serializability - intuition

- **Alternative definition: a schedule S is conflict serializable if**
 - You are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A)	W(A)		R(B)	W(B)	
		R(A)	W(A)		
				R(B)	W(B)

R(A)	W(A)		R(B)	W(B)	
		R(A)	W(A)		
				R(B)	W(B)

Conflict serializability - intuition

- **Alternative definition: a schedule S is conflict serializable if**
 - You are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A)	W(A)		R(B)	W(B)	
		R(A)	W(A)		
				R(B)	W(B)

R(A)	W(A)		R(B)		W(B)	
		R(A)		W(A)		
					R(B)	W(B)

Conflict serializability - intuition

- **Alternative definition: a schedule S is conflict serializable if**
 - You are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A)	W(A)		R(B)	W(B)	
		R(A)	W(A)		
				R(B)	W(B)

R(A)	W(A)		R(B)	W(B)	
		R(A)			
			W(A)	R(B)	W(B)

Conflict serializability - intuition

- **Alternative definition: a schedule S is conflict serializable if**
 - You are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A)	W(A)		R(B)	W(B)	
		R(A)	W(A)		
				R(B)	W(B)

R(A)	W(A)	R(B)		W(B)		
		R(A)		W(A)	R(B)	W(B)

Conflict serializability - intuition

- **Alternative definition: schedule S is conflict serializable if**
 - You are able to transform S into a serial schedule by swapping **consecutive non-conflicting** operations of different transactions
- *Example*

R(A) W(A)	R(B) W(B)
R(A) W(A)	R(B) W(B)

R(A) W(A) R(B) W(B)

R(A) W(A) R(B) W(B)

Conflict serializability (continued)

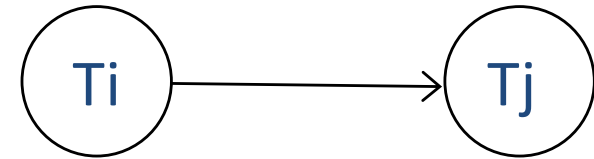
- Here's another example:

$R(A)$ $W(A)$
 $R(A)$ $W(A)$

- Conflict Serializable or not?

NOT!

Dependency graph



- Dependency graph:
 - One node per transaction
 - Edge from T_i to T_j when:
 - Some operation O_i of T_i conflicts with an operation O_j of T_j and
 - O_i appears earlier in the schedule than O_j
- *Theorem: Schedule is conflict serializable if and only if its dependency graph is acyclic*

Example

- A schedule that is not conflict serializable

T1: R(A), W(A)
T2:

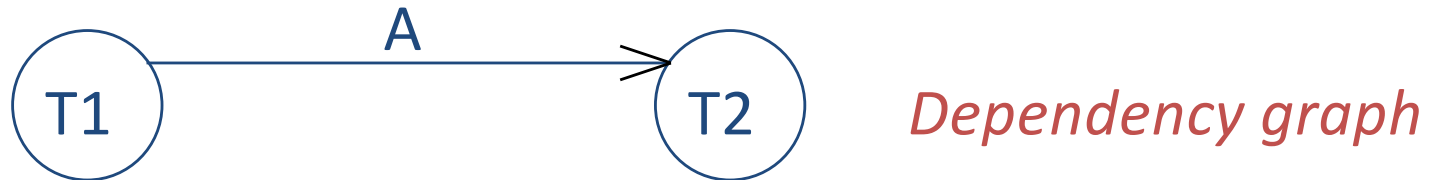


Dependency graph

Example

- A schedule that is not conflict serializable

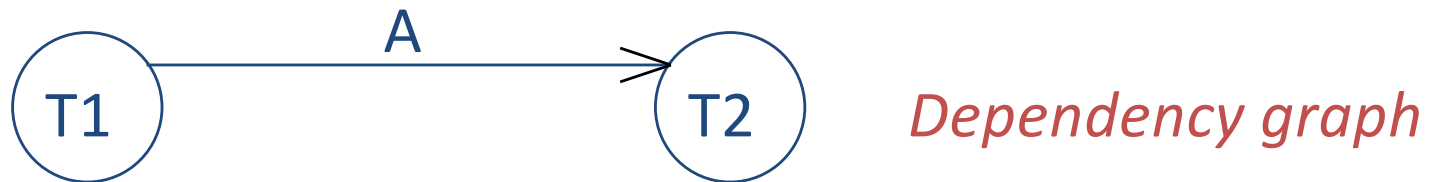
T1: R(A), W(A),
T2: R(A)



Example

- A schedule that is not conflict serializable

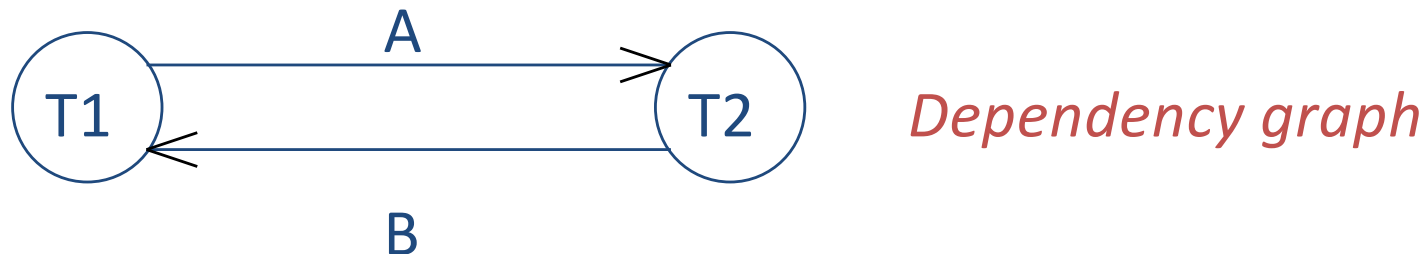
T1: R(A), W(A),
T2: R(A), W(A), R(B), W(B)



Example

- A schedule that is not conflict serializable

T1: R(A), W(A),	R(B)
T2: R(A), W(A), R(B), W(B)	



- The cycle in the graph reveals the problem. The output of T1 depends on T2, and vice versa

Quick quizz

For 2 conflicting operations, what is true?

1. At least one is a write
2. At least one is a read
3. They are on the same object
4. They are from the same transaction

Quick quizz

Time



T1	W(B)						R(E)		
T2		R(C)						W(B)	
T3				W(C)					W(E)
T4			R(E)		R(B)	W(B)			

- dependency graph for this schedule ?
- is it conflict serializable?
- If yes, give the equivalent serial schedules

Alternative: view serializability

- Alternative (weaker!) notion of serializability: schedules S1 and S2 are **view equivalent** if
 - *Same initial reads*: If T_i reads initial value of A in $S1$, then T_i also reads initial value of A in $S2$
 - *Same dependent reads*: If T_i reads value of A written by T_j in $S1$, then T_i also reads value of A written by T_j in $S2$
 - *Same winning writes*: If T_i writes final value of A in $S1$, then T_i also writes final value of A in $S2$
- Basically, allows all conflict serializable schedules + “blind writes” (write w/o read)



Notes on serializability definitions

- View serializability allows (a few) more schedules than conflict serializability does
 - But difficult to enforce efficiently
- **Neither definition allows all schedules** that are actually serializable
 - Because they don't understand the meanings of the operations or the data
- In practice, conflict serializability is used, because it can be enforced efficiently
 - To allow more concurrency, some special cases do get handled separately

Schedules with aborted transactions

- When a transaction aborts, the recovery manager undoes its updates
- But some of its updates may have affected other transactions !

Schedules with aborted transactions

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
	Commit
Abort	

- Pb: cannot abort T1 because cannot undo T2

Recoverable schedules

Definition: A schedule is recoverable if whenever a transaction T commits, all transactions who have written elements read by T have already committed

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
	Commit
Abort	

Not recoverable

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
Abort	
	Commit

Recoverable

Cascading aborts

- If a transaction T_1 aborts, then we need to abort any transaction T_2 that has read an element written by T_1
- A schedule is said to **avoid cascading aborts** if whenever a transaction read an element, the transaction that has **last** written it has already committed.

Schedules with cascading aborts

T1	T2
R(A)	
W(A)	
	R(A)
	W(A)
	R(B)
	W(B)
...	
	...

With cascading aborts

T1	T2
R(A)	
W(A)	
Commit	
	R(A)
	W(A)
	R(B)
	W(B)
	...

Without cascading aborts

How to enforce “good” schedules

LOCKING

Two phase locking (2PL)

Idea: find rules such that “good schedules” are guaranteed if all transactions obey them.

- The most common approach for enforcing conflict serializability
- A bit “pessimistic”
 - Sets locks for fear of conflict... some cost here
 - Alternative schemes “optimistically” let transactions move forward, and either abort or “repair” them when conflicts are detected

Locking:

- each element (?) has an unique lock
- each transaction must acquire that lock before reading / writing that element
- transactions release locks (eventually)

Two phase locking (2PL)

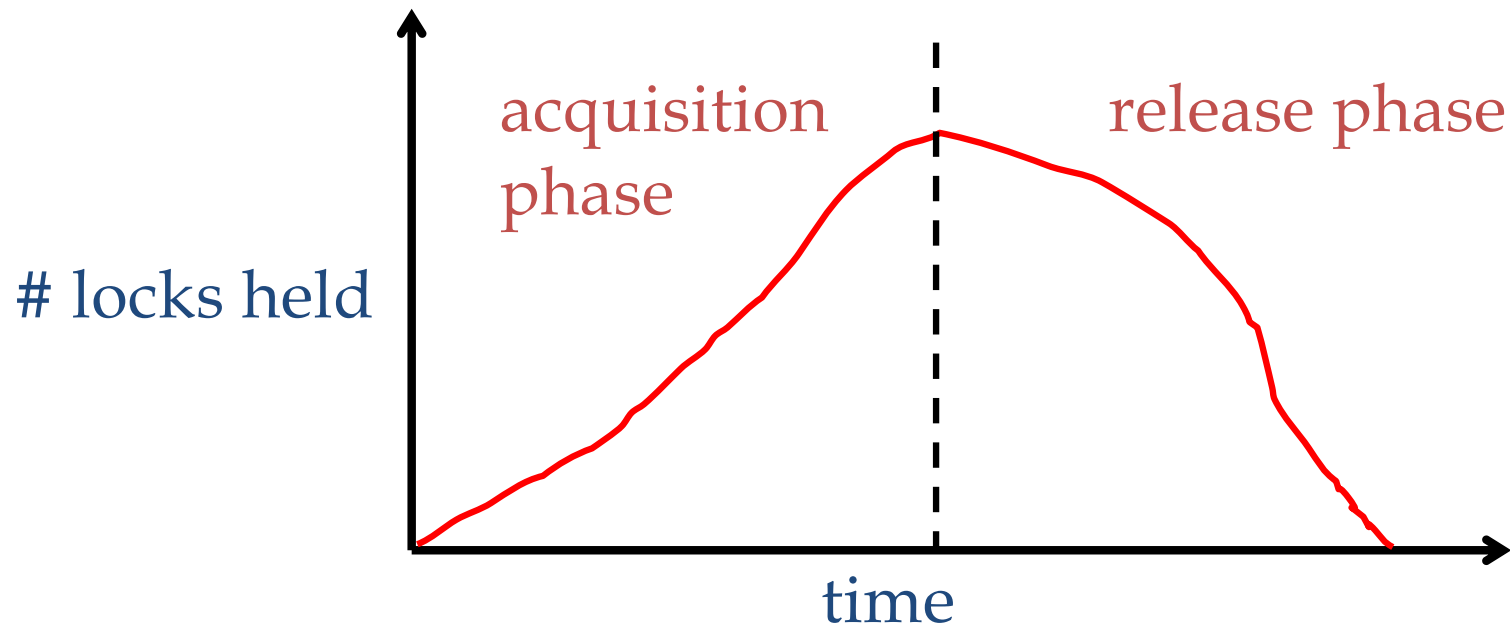
Lock
Compatibility
Matrix

	S	X
S	✓	—
X	—	—

2PL's rules:

- A transaction must obtain a **S** (shared) lock before reading
- A transaction must obtain an **X** (exclusive) lock before writing
- A transaction cannot get new locks after releasing any locks

Two phase locking (2PL)



- 2PL guarantees conflict serializability (why?)
- But, does not prevent **cascading aborts**, does not ensure **recoverable schedules**

Why 2PL guarantees conflict serializability

- When a committing transaction has reached the end of its acquisition phase...
 - Call this the "lock point"
 - At this point, it has *everything it needs* locked...
 - ... and any conflicting transactions either:
 - started release phase before this point
 - are blocked waiting for this transaction
- Hence visibility of actions of two conflicting transactions are ordered by their lock points
- Order of lock points gives us an equivalent serial schedule!

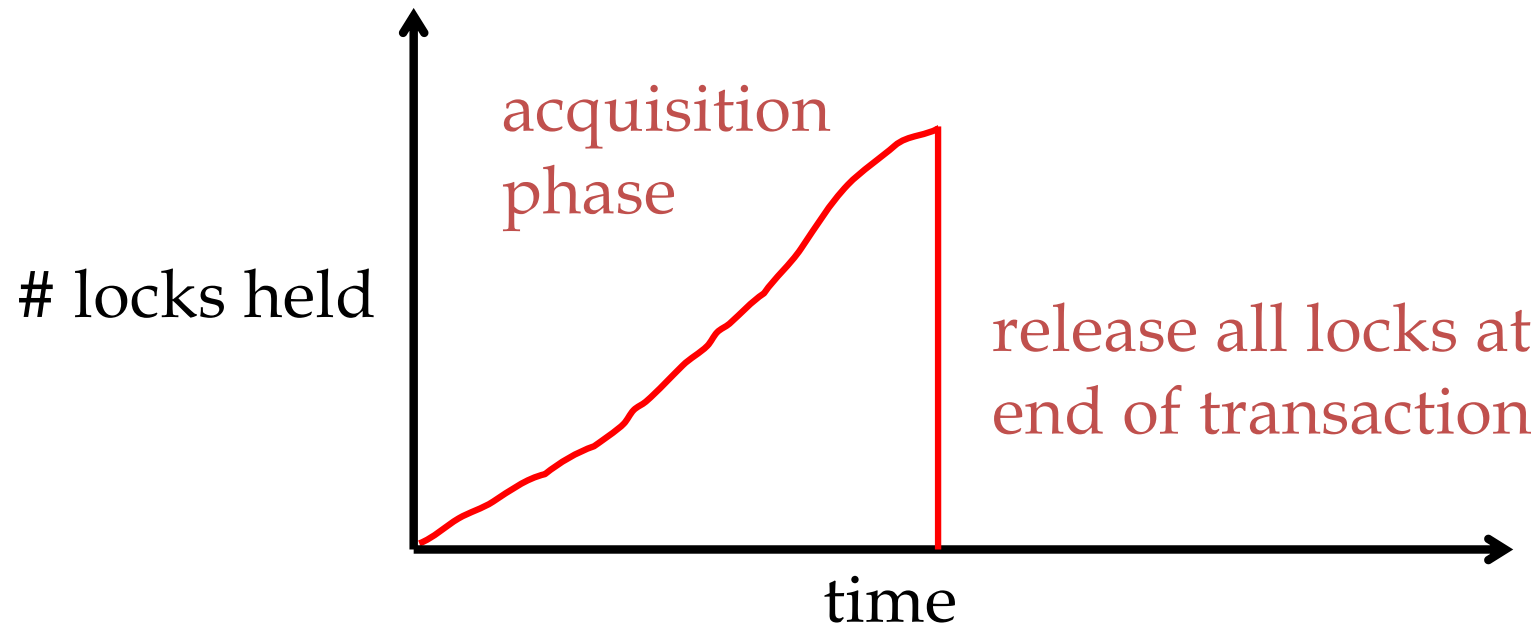
Strict two-phase locking (2PL)

- Problem: cascading aborts
- Example: rollback of T1 requires rollback of T2 !

T1: R(A), W(A)	Abort
T2: R(A), W(A)	

- Strict two-phase locking (strict 2PL) protocol:
 - Same as 2PL, except
 - All locks released together when transaction completes
 - (i.e.) either
 - transaction has committed (commit record on disk)
 - or
 - transaction has aborted and rollback is complete

Strict two phase locking (2PL)



Quick quizz

- The two phases of 2PL are:
 - A. Prepare and commit
 - B. Acquisition and release
 - C. Optimization and execution
- Which is avoided by 2PL?
 - A. Isolation problems
 - B. Cascading abort
 - C. Cyclic dependency graphs
- Which is avoided by *strict* 2PL?
 - A. Isolation problems
 - B. Cascading abort
 - C. Cyclic dependency graphs

Non-2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A)	
	Lock_S(A)
A: = A-50	
Write(A)	
Unlock(A)	
	Read(A)
	Unlock(A)
	Lock_S(B)
Lock_X(B)	
	Read(B)
	Unlock(B)
	PRINT(A), PRINT(B), PRINT(A+B)
Read(B)	
B := B + 50	
Write(B)	
Unlock(B)	

Non-2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A): (A=1000)	
	Lock_S(A)
A: = A-50 (A=950)	
Write(A) A=950	
Unlock(A)	
	Read(A) (A = 950)
	Unlock(A)
	Lock_S(B)
Lock_X(B)	
	Read(B) (B=2000)
	Unlock(B)
	PRINT(A), PRINT(B), PRINT(A+B)
Read(B) (B=2000)	
B := B + 50 (B=2050)	
Write(B) B=2050	
Unlock(B)	

Output: 950, 2000, 2950

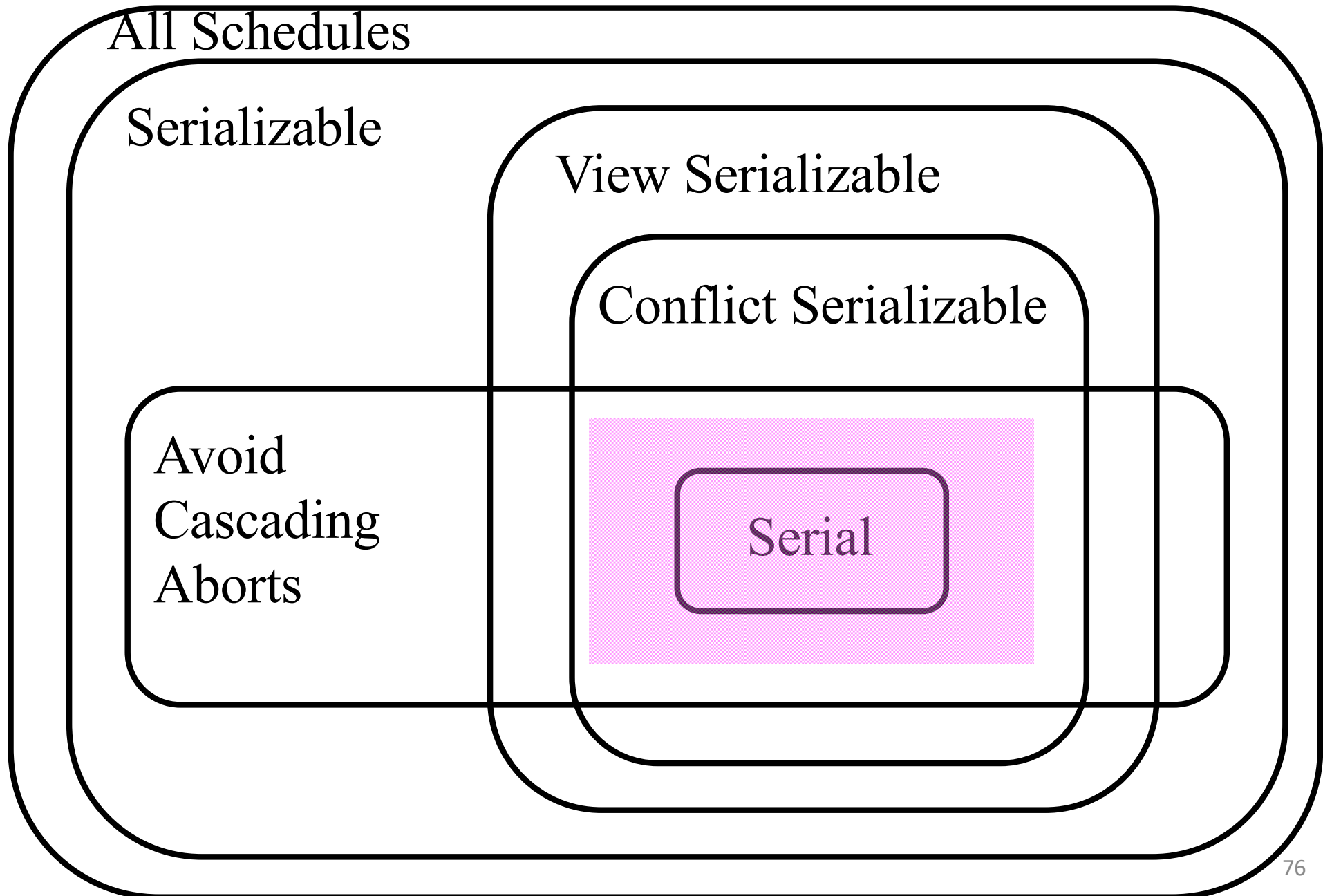
2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A)	
A: = A-50	
Write(A)	
Lock_X(B)	
Unlock(A)	
	Lock_S(A)
	Read(A)
Read(B)	
B := B +50	
Write(B)	
Unlock(B)	
	Lock_S(B)
	Unlock(A)
	Read(B)
	Unlock(B)
Output: 950, 2050, 3000	PRINT(A), PRINT(B), PRINT(A+B)

Strict 2PL, A = 1000, B = 2000, Output = ?

T1	T2
Lock_X(A)	
Read(A)	
	Lock_S(A)
A: = A-50	
Write(A)	
Lock_X(B)	
Read(B)	
B := B + 50	
Write(B)	
Unlock(A)	
Unlock(B)	
	Read(A)
	Lock_S(B)
	Read(B)
	PRINT(A), PRINT(B), PRINT(A+B)
	Unlock(A)
Output: 950, 2050, 3000	Unlock(B)

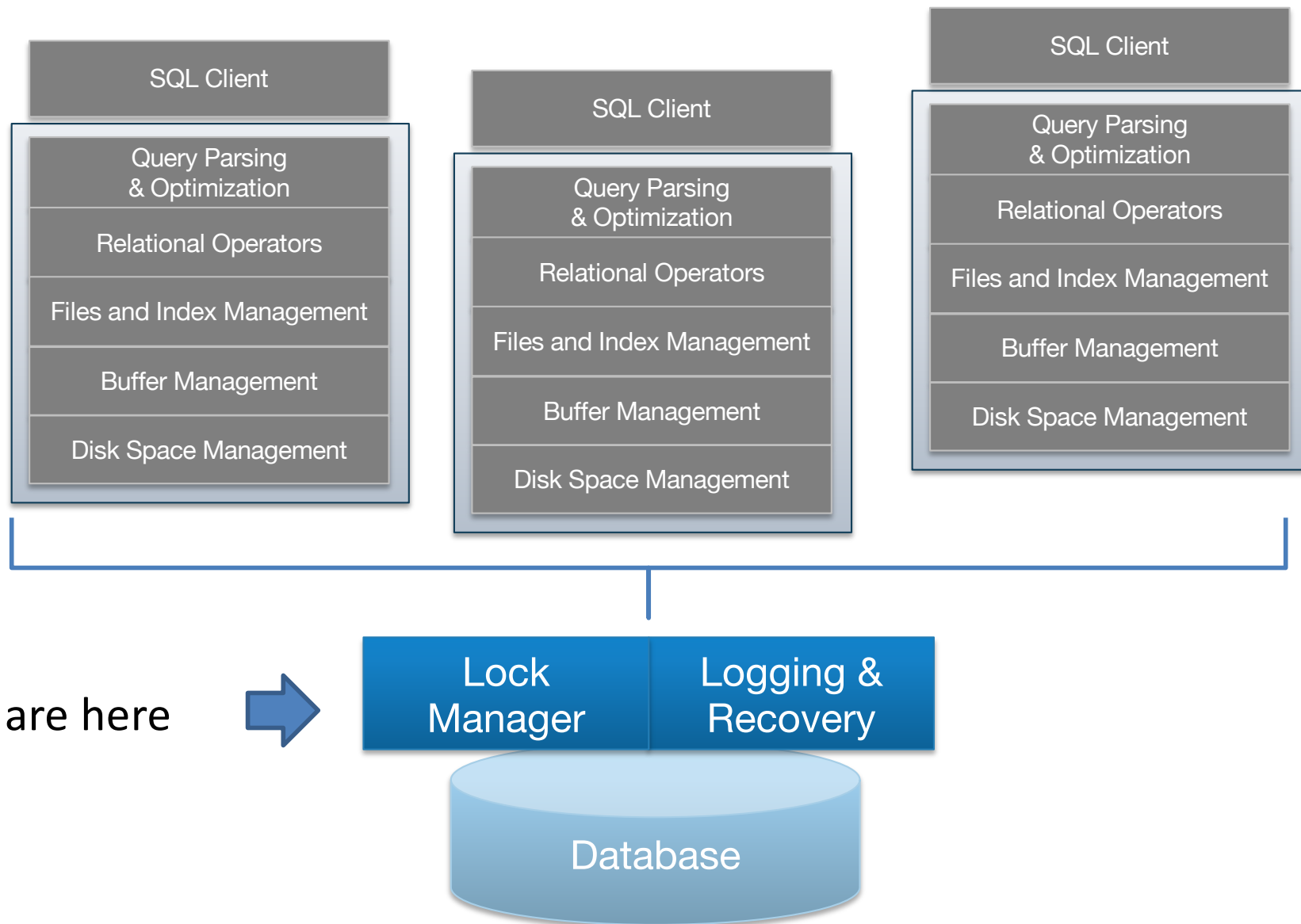
Which schedules does Strict 2PL allow?



Locks in applications?

- Applications typically do not acquire a locks manually (i.e., explicit SQL commands).
- We may provide the DBMS with hints to help it to improve concurrency
 - Update a tuple after reading it (SELECT ... FOR UPDATE)
 - Skip any tuple that is locked
- Explicit locks may be useful when doing major changes to the DB

Architecture



Lock management

- Lock and unlock requests handled by Lock Manager
- LM maintains a hash table, keyed on names of objects being locked.
- LM keeps an entry for each currently held lock
- Entry contains
 - **Granted set:** set of transactions currently granted access to the lock
 - **Lock mode:** type of lock held (shared or exclusive)
 - **Wait queue:** queue of lock requests

	Granted Set	Mode	Wait Queue
A	{T1, T2}	S	T3(X) $\leftarrow$ T4(X)
B	{T6}	X	T5(X) $\leftarrow$ T7(S)

Lock management (continued)

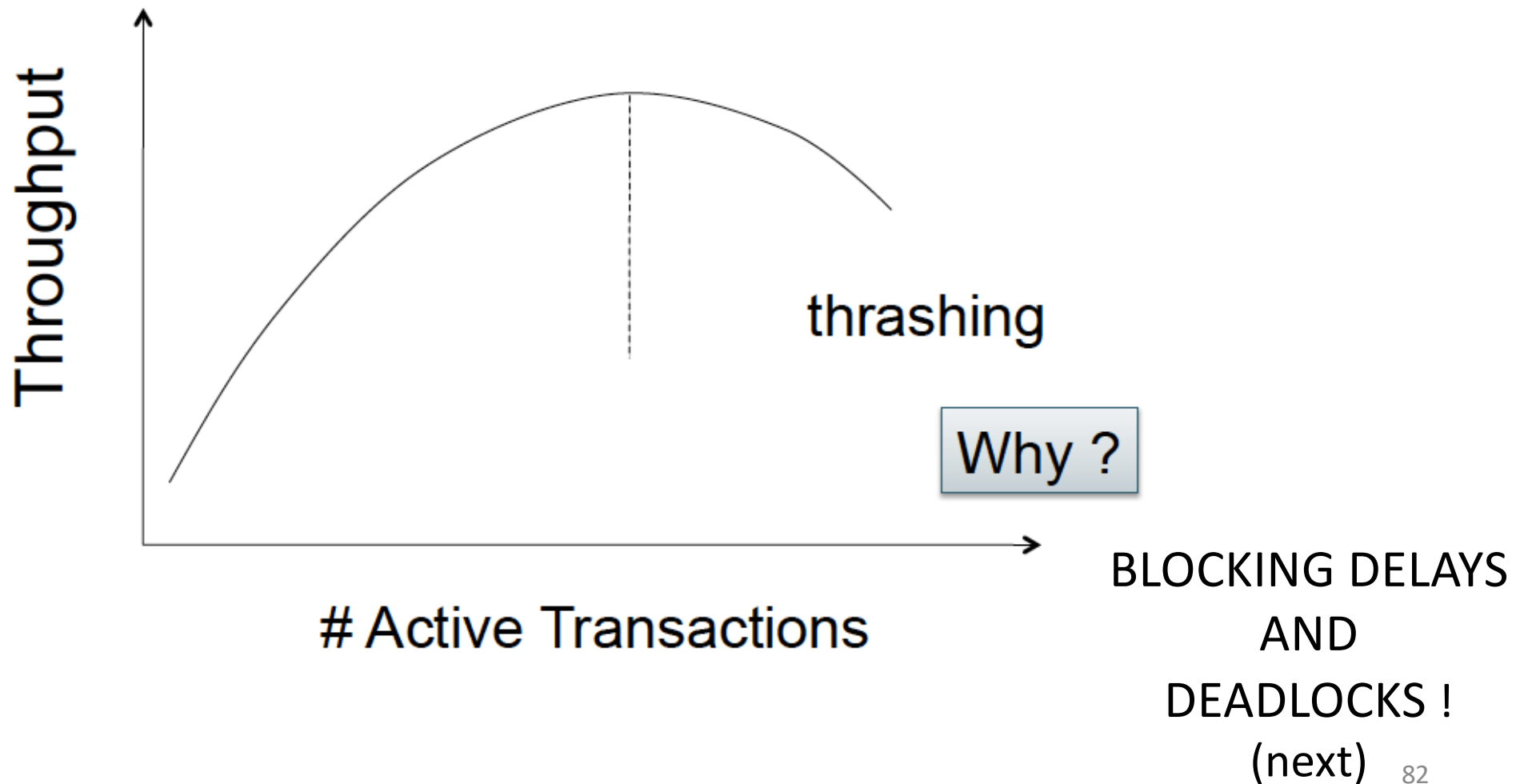
- **When lock request arrives:**
 - Does any other transaction hold a conflicting lock?
 - If no, put the requester into “granted set” and let them proceed
 - If yes, put requester into wait queue (typically FIFO)
- **Lock upgrade:**
 - Transaction with shared lock can request to upgrade to exclusive

	Granted set	Mode	Wait queue
A	{T1, T2}	S	T2(X) ← T3(X) ← T4(X)
B	{T6}	X	T5(X) ← T7(S)

Quick quizz

- Which of the following can be a state of the “granted set” and “mode” for a lock:
 - A. 2 different transactions, X mode
 - B. 1 transaction, X mode
 - C. 2 different transactions, S mode
 - D. 1 transaction in X mode, 1 in S mode

LM performance



Example

Lock_X(A)	
	Lock_S(B)
	Read(B)
	Lock_S(A)
Read(A)	
A: = A-50	
Write(A)	
Lock_X(B)	

Final lock table state:

A:

- X lock held by T1
- wait queue = [T2 wants S]

B:

- S lock held by T2
- wait queue = [T1 wants X]

Pb: T1 and T2 are waiting for each other!

83

SQL isolation levels (lock-based)

- Read Uncommitted
 - Idea: can read dirty data
 - Implementation: no locks on read
- Read Committed
 - Idea: only read committed items
 - Implementation: can unlock immediately after read
- Repeatable Read
 - Idea: if you read an item twice in a transaction, you see the same committed version
 - Implementation: hold read locks until end of transaction
 - No phantom protection
- Serializable (avoids all problems / anomalies) – strict 2PL +



Paranthesis about “phantoms”

- Suppose you query for sailors with rating between 10 and 20, using a B+ tree
 - Tuple level locks in the Heap File
- I insert a new sailor “John Doe”, with rating 12
- You run your query again
 - Problem: a phantom sailor
 - Reason: Serializability assumed a static DB
- What we want: lock the logical range 10-20
 - Imagine that lock table!
- What is done: set locks on indexes cleverly

Phantom – definition

- “Phantom”: tuple that is invisible during part of a transaction execution but not invisible during the entire execution

Key facts:

- In a static database:
 - Conflict serializability implies serializability
- In a dynamic database, this may fail due to phantoms
- Strict 2PL guarantees conflict serializability, but not serializability
- Expensive ways of dealing with phantoms (strict 2PL+index-tree protocol; details beyond our scope)
 - Serializable transactions are very expensive !

Isolation levels in SQL

“Dirty reads”

SET TRANSACTION ISOLATION LEVEL READ
UNCOMMITTED

“Committed reads”

SET TRANSACTION ISOLATION LEVEL READ COMMITTED

“Repeatable reads”

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ

Serializable transactions

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE

Isolation level: dirty reads

- “Long duration” WRITE locks
 - Strict 2PL
- No READ locks
 - Note: read-only transactions are never delayed

Possible pbs: dirty and inconsistent reads

Isolation level: read committed

- “Long duration” WRITE locks
 - Strict 2PL
- “Short duration” READ locks
 - Only acquire lock while reading (so not 2PL...)
- Possible pbs: inconsistent (unrepeatable) reads (when reading same element twice, may get two different values)

Isolation level: repeatable read

- “Long duration” WRITE locks
 - Strict 2PL
- “Long duration” READ locks
 - Strict 2PL
- This is not serializable yet !

Isolation level: serializable

- “Long duration” WRITE locks
 - Strict 2PL
- “Long duration” READ locks
 - Strict 2PL
- ... and deals with phantoms too

Isolation levels vs. anomalies

	Dirty Read	Unrepeatable Read	Phantom
READ UNCOMMITTED	Maybe	Maybe	Maybe
READ COMMITTED	No	Maybe	Maybe
REPEATABLE READS	No	No	Maybe
SERIALIZABLE	No	No	No

No lost updates at all levels !



DEADLOCKS (NOT TO BE CONFUSED WITH DREADLOCKS)

Deadlocks

- Deadlock: Cycle of transactions waiting for locks to be released by each other
- Three ways of dealing with deadlocks:
 - Prevention
 - Avoidance
 - Detection and resolution
- Many systems simplify and use timeouts
 - What are the dangers with this approach ?

Deadlock scenarios

- They can just happen (unavoidable)

	Granted Set	Mode	Wait Queue
A	{T1}	S	T2(X)
B	{T2}	X	T1(S)

- Bad implementation of lock upgrade (avoidable!)

	Granted Set	Mode	Wait Queue
A	{T1, T2}	S	T3(X) ← T4(X) ← T2(X)

- Simultaneous lock upgrade (unavoidable)

	Granted Set	Mode	Wait Queue
A	{T1, T2}	S	T2(X) ← T1(X) ← T3(X) ← T4(X)

Deadlock prevention

- Common technique in operating systems
- Standard approach: resource ordering (Havender rule)
 - Screen < Network Card < Printer
- Why is this problematic for transactions in a DBMS ?

Deadlock avoidance

- **Assign priorities** based on age, e.g. (now – start_time).
- Say T_i wants a lock that T_j holds. Two possible policies:
 - Wait-Die: If T_i has higher priority, T_i waits for T_j ; else T_i aborts
 - Wound-Wait: If T_i has higher priority, T_j aborts; else T_i waits
- Read each of these like a ternary operator (C, java, etc)



Quick quizz

- Wait-Die

- If T_{old} wants a lock from T_{young} , T_{old} does what ?
- If T_{young} wants a lock from T_{old} , T_{young} does what

- Wound-Wait

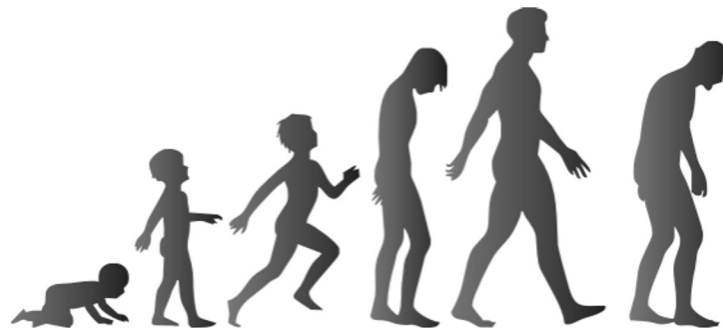
- If T_{old} wants a lock from T_{young} , T_{young} does what ?
- If T_{young} wants a lock from T_{old} , T_{young} does what ?

Quick quizz

- Wait-Die
 - If T_{old} wants a lock from T_{young} , T_{old} **waits**
 - If T_{young} wants a lock from T_{old} , T_{young} **aborts**
- Wound-Wait
 - If T_{old} wants a lock from T_{young} , T_{young} **aborts**
 - If T_{young} wants a lock from T_{old} , T_{young} **waits**

Deadlock avoidance: analysis

- Q: Why do these schemes guarantee no deadlocks ?
- Priority usually based on transaction's age
 - $(\text{now} - \text{timestamp})$
 - sometimes based on a measure of resource consumption, e.g. #locks held
- Important detail: If a transaction re-starts, make sure it gets its original timestamp. Why?



Deadlock detection

- Create and maintain a “waits-for” graph
- Periodically check for cycles in a graph

Deadlock detection (continued)

Example:

T1:

T2:

T3:

T4:



Deadlock detection (continued)

Example:

T1: S(A)

T2:

T3:

T4:



Deadlock detection (continued)

Example:

T1: S(A) S(D)

T2:

T3:

T4:



Deadlock detection (continued)

Example:

T1: S(A) S(D)

T2: X(B)

T3:

T4:



Deadlock detection (continued)

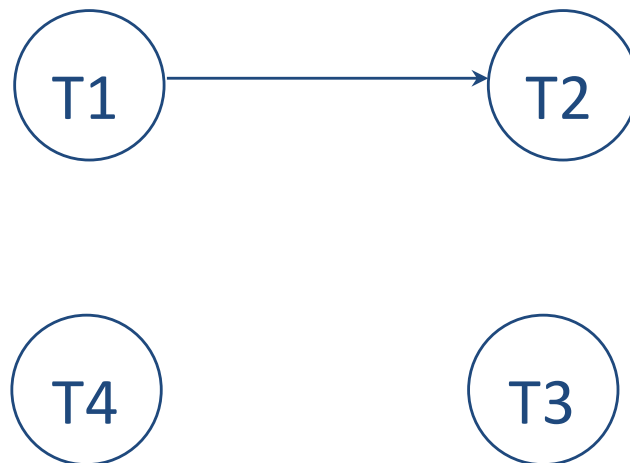
Example:

T1: S(A) S(D) S(B)

T2: X(B)

T3:

T4:



Deadlock detection (continued)

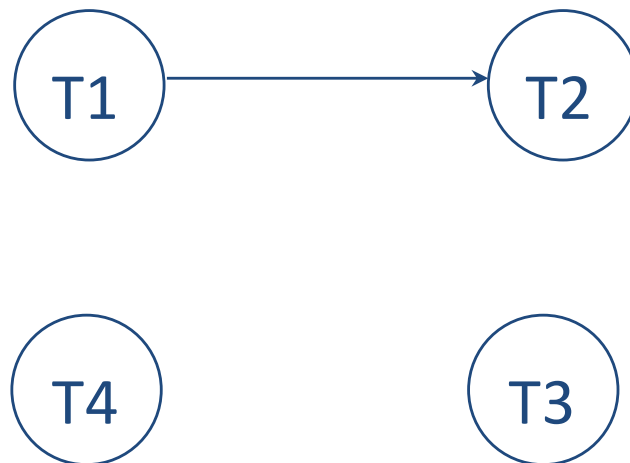
Example:

T1: S(A) S(D) S(B)

T2: X(B)

T3: S(D)

T4:



Deadlock detection (continued)

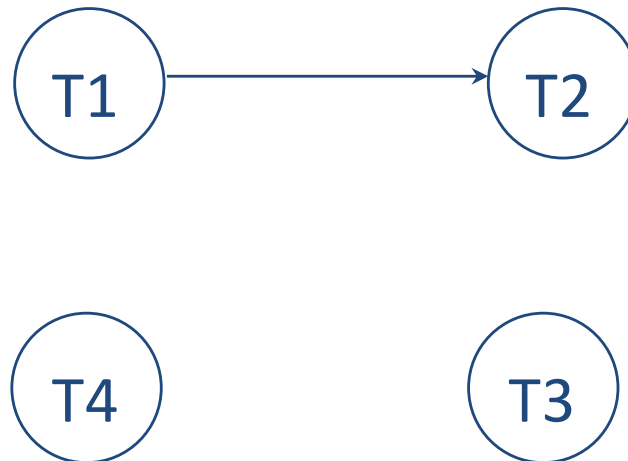
Example:

T1: S(A) S(D) S(B)

T2: X(B)

T3: S(D), S(C)

T4:



Deadlock detection (continued)

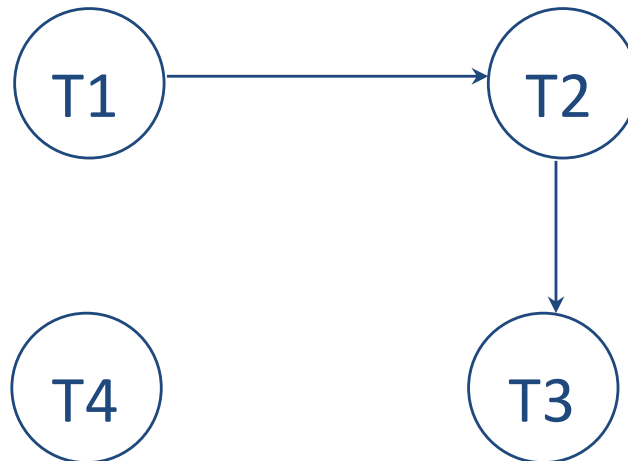
Example:

T1: S(A) S(D) S(B)

T2: X(B) X(C)

T3: S(D) S(C)

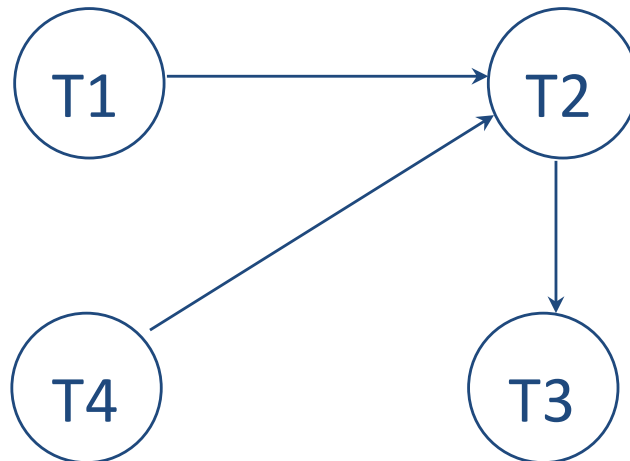
T4:



Deadlock detection (continued)

Example:

T1:	S(A)	S(D)	S(B)	
T2:		X(B)		X(C)
T3:			S(D)	S(C)
T4:				X(B)



Deadlock detection (continued)

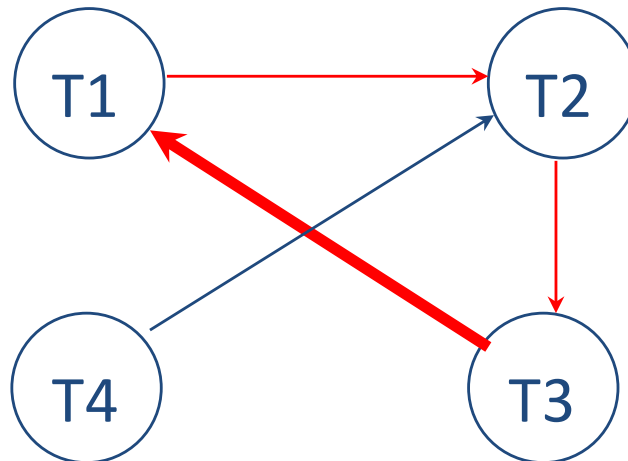
Example:

T1: S(A) S(D) S(B)

T2: X(B) X(C)

T3: S(D) S(C) X(A)

T4: X(B)



Deadlock !

- **T1, T2, T3 are deadlocked**
 - Doing no good, and holding locks
- **T4 still cruising**
- **In the background, run a deadlock detection algorithm**
 - Periodically extract the waits-for graph
 - Find cycles
 - “Shoot” a transaction on the cycle (find victim)
- **Empirical fact**
 - Most deadlock cycles are small (2-3 transactions)

Quick quizz

- True or False:
 - Deadlocks sometimes resolve themselves: any one transaction that chooses to commit breaks the cycle
 - A waits-for graph may have more than one cycle
 - A waits-for graph may have cycles within cycles
 - The choice of victim can affect overall performance

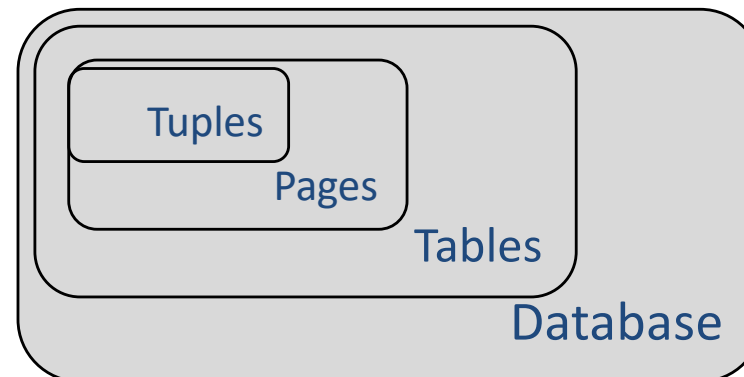
Quick quizz

Draw the waits-for graph and determine whether or not this schedule is deadlocked

T1	X(A)			S(B)			
T2		X(B)	S(A)				
T3					X(C)	S(A)	
T4							S(C)

Summary

- Correctness criterion for isolation is “serializability”
 - In practice, we use “conflict serializability” which is somewhat more restrictive but easy to enforce
- Two Phase Locking (2PL) and Strict 2PL: Locks implement the notions of conflict directly – used in almost every DBMS!
 - The lock manager keeps track of the locks issued
 - Deadlocks may arise; can either be prevented or detected
- More to the story
 - Optimistic CC (Snapshot, Timestamp)
 - Tree protocol (index “latching” – quick locks on index nodes) to avoid phantoms
 - **Multi-granularity locking and intention locks.**



Isolation levels

Database	Default	Maximum
Action Ingres 10.0/10S	S	S
Aerospike	RC	RC
Akiban Persistit	SI	SI
Clustrix CLX 4100	RR	RR
Greenplum 4.1	RC	S
IBM DB2 10 for z/OS	CS	S
IBM Informix 11.50	Depends	S
MySQL 5.6	RR	S
MemSQL 1b	RC	RC
MS SQL Server 2012	RC	S
NuoDB	CR	CR
Oracle 11g	RC	SI
Oracle Berkeley DB	S	S
Oracle Berkeley DB JE	RR	S
Postgres 9.2.2	RC	S
SAP HANA	RC	SI
ScaleDB 1.02	RC	RC
VoltDB	S	S
RC: read committed, RR: repeatable read, SI: snapshot isolation, S: serializability, CS: cursor stability, CR: consistent read		

Table 2: Default and maximum isolation levels for ACID and NewSQL databases as of January 2013 (from [9]).

Extra slides

OPTIMISTIC CONCURRENCY CONTROL